Crypto Pulse

Real-Time Market Anomaly Radar

Project Handbook

Setup guide, how it works, file-by-file tour, design decisions, interview preparation, and the roadmap to industry level.

Written for someone new to tech: nothing is assumed.

Repository: github.com/jayasinghania/crypto-anomaly-radar | October 2026

Contents

Start here: the project in one minute 4

What this project is NOT . . . 4

Why this project is valuable . . . 4

The tools, in plain words . . . 5

1. Setup: get it running on your PC 5

1.1 Install these first . . . 5

1.2 Git in plain English . . . 6

1.3 One-time GitHub setup . . . 6

1.4 Download the code . . . 6

1.5 Docker for beginners . . . 7

1.6 Option A (recommended): run everything with Docker . . . 7

1.7 Option B: run the Python code directly (for developers) . . . 8

1.8 Run the tests . . . 8

1.9 Your everyday git routine . . . 9

1.10 Troubleshooting: the errors we actually hit . . . 9

2. The flow of the project 10

2.1 The big picture . . . 10

2.2 The life of one price . . . 11

2.3 The database: four tables . . . 11

2.4 Three programs running at the same time . . . 12

2.5 Redis pub/sub: the radio station . . . 12

2.6 Anomaly detection with a z-score . . . 13

2.7 The forecast (and why it is honest) . . . 13

2.8 How Docker Compose wires it together . . . 14

3. A tour of every file 15

3.1 Files in the project root . . . 16

3.2 app/ : the heart of the project . . . 16

3.3 migrations/ : version control for the database . . . 18

3.4 tests/ : the safety net . . . 19

4. Design and implementation decisions 20

4.1 Structure and reliability . . . 20

4.2 Data and storage . . . 21
4.3 Analytics and real-time . . . 21
4.4 Packaging, testing and delivery . . . 22
4.5 Bugs we hit, and what each one taught . . . 23
5. Interview questions and model answers **24**
Numbers you can quote (all true and reproducible) . . . 24
A. The project overall . . . 24
B. Architecture . . . 25
C. Python and async . . . 25
D. Database . . . 26
E. Real-time . . . 26
F. Statistics and the forecast . . . 27
G. Testing, Docker and CI . . . 27
H. Honesty, limits and growth . . . 28
6. Future scope: toward an industry-level system **29**
6.1 Where we are versus industry level . . . 29
6.2 A realistic roadmap . . . 29
6.3 Cautions for the real world . . . 30
6.4 A learning path if you are new . . . 31
Appendix A: Glossary **32**
Appendix B: Command cheat sheet **34**
Appendix C: Addresses and defaults **35**

Start here: the project in one minute

Crypto Pulse (the GitHub repository is called **crypto-anomaly-radar**) is a small system that watches cryptocurrency prices live and raises a flag when a price does something unusual.

In plain English: every 30 seconds the program asks a public website (CoinGecko) *"what are Bitcoin, Ethereum, Solana, Cardano and Dogecoin worth right now?"* and saves each answer in a database. Every minute, a second program looks at the last 20 minutes of prices for each coin and asks *"is the newest price strangely far from what is normal for this coin?"* If yes, it raises an **anomaly** flag. Everything is pushed live to a web page, so you watch prices move without ever refreshing. The project also makes a simple 5-minute forecast and honestly reports whether that forecast is any better than just assuming the price will not change.

> **Think of it as a smoke alarm for prices**
> A smoke alarm does not predict fires. It notices when the air is abnormal and tells you immediately. Crypto Pulse does the same for prices: it does not predict the market, it notices when the market is behaving unusually.

What this project is NOT

- **Not trading advice.** Nothing here tells anyone to buy or sell.
- **Not a prediction machine.** The forecast is a learning exercise, and our own tests show it does not beat the simplest possible guess over one minute.
- **Not (yet) a production system.** It runs on one computer. Section 6 explains what a real company would add.

Why this project is valuable

It walks through the **entire journey of data**: collect it, store it, analyse it, serve it, and display it, plus the habits professional teams expect: automated tests, Docker, and continuous integration (CI). Each of these words is explained in this handbook.

The tools, in plain words

Tool	What it is, simply	Where it is used
Python 3.12 / 3.13	The programming language everything is written in	All of the app
asyncio + httpx	asyncio lets a program wait for slow things (like the internet) without freezing; httpx sends web requests	Fetching prices
PostgreSQL	A database: organised tables of data stored safely on disk	Storage
SQLAlchemy	Lets Python talk to the database using Python classes instead of raw SQL	Storage
Alembic	Version control for the database's structure (called migrations)	migrations/ folder
Redis	A very fast in-memory message board (pub/sub: publish and subscribe)	Live updates
FastAPI + Uvicorn	FastAPI builds web APIs; Uvicorn is the server that runs it	app/api/
WebSocket	A phone call between browser and server that stays open (normal web requests are like letters)	Live dashboard
Chart.js, HTML, JS	Draws the live charts in your browser	app/dashboard/
Docker + Compose	Packages the app and its helpers so they run identically on any computer	Dockerfile, docker-compose.yml
pytest	Runs automatic tests that check the code still works	tests/ folder
GitHub Actions	A robot that runs the tests every time someone changes the code	.github/workflows/ci.yml

1. Setup: get it running on your PC

Follow this section top to bottom. Expect about 45 minutes the first time, most of it waiting for downloads.

1.1 Install these first

Tool	Why you need it	Check that it worked
Git	Downloads and shares the code	`git --version`
Python 3.12 or 3.13	Runs the code. During install, tick **"Add python.exe to PATH"**	`python --version`
Docker Desktop	Runs Postgres, Redis and the whole project in containers	`docker --version` and `docker compose version`
VS Code (optional)	A friendly editor to read the code	Opens without errors
GitHub account	Needed to access the repository	You can log in at github.com

> **Docker Desktop on Windows**
> The installer will ask to use **WSL 2**: say yes. If it complains about virtualization, it must be switched on in your PC's BIOS settings. Restart the PC after installing, then **open Docker Desktop and wait until it says the engine is running** before using any docker command.

1.2 Git in plain English

Git is a time machine for code. It is like Google Docs version history, except *you* decide when to save a version and you attach a note describing it. GitHub is the website that stores the shared copy.

Word	Meaning
Repository (repo)	The project folder, plus its full history
Commit	A saved snapshot with a message, e.g. "Add forecast endpoint"
Branch	A separate line of work. **main** is the stable version; you do your work on your own branch
Remote / origin	The copy of the repo that lives on GitHub
Clone	Download the repo to your PC for the first time
Pull	Download new changes from GitHub into your copy
Push	Upload your commits to GitHub
Pull request (PR)	A request to merge your branch into main; a teammate reviews it and automatic tests run

1.3 One-time GitHub setup

1. **Accept the invitation.** You were added as a collaborator. Open github.com/jayasinghania/crypto-anomaly-radar/invitations and click Accept. Until you do, every push fails with a "403 permission denied" error.
2. **Tell git who you are** (use your real name and email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

3. Create a personal access token. GitHub no longer accepts your account password in the terminal. A token is a long generated password for exactly this purpose. On GitHub: profile photo, Settings, Developer settings, Personal access tokens, **Tokens (classic)**, Generate new token (classic). **Tick the "repo" box**, generate, and copy the token immediately (GitHub shows it only once).

> **The mistake that cost us an hour**
> A token with **no boxes ticked** logs you in fine but cannot push, and the error looks like a permissions problem (403). Always tick **repo**. When git asks for a password, paste the token, not your GitHub password.

1.4 Download the code

```
cd D:\projects
git clone https://github.com/jayasinghania/crypto-anomaly-radar.git
cd crypto-anomaly-radar
```

A login window or prompt appears the first time: use your GitHub username and the token. Windows remembers it afterwards. Run `dir` (Windows) or `ls` (Mac/Linux): you should see `Dockerfile`, `docker-compose.yml`, `requirements.txt` and an `app` folder.

1.5 Docker for beginners

Imagine every program needs a perfect kitchen: the right oven, the right tools. Instead of building that kitchen on your PC, Docker gives you a **sealed box with the kitchen already inside**. That is why the project behaves identically on your PC, your friend's PC, and the CI robot.

Docker word	Everyday comparison	In this project
Image	A frozen meal kit: instructions plus ingredients	postgres:16, redis:7-alpine, and our own image built from the Dockerfile
Container	The meal being cooked right now from that kit	Each running service: database, redis, api, scheduler, worker
Volume	A fridge that keeps food even if you replace the pot	pgdata: your database files survive restarts
Compose file	The menu: which dishes, how they connect	docker-compose.yml starts everything with one command

Useful commands (run them from the repo folder):

Command	What it does
`docker compose up -d --build`	Build the images and start everything in the background
`docker compose ps -a`	Show every service and its state (including ones that finished)
`docker compose logs -f scheduler`	Watch one service's messages live (Ctrl+C to stop watching)
`docker compose down`	Stop and remove containers. **Your data is kept.**
`docker compose down -v`	**Danger:** also deletes the database volume, wiping all stored prices

1.6 Option A (recommended): run everything with Docker

This is the easiest path and needs no Python setup at all.

1. Make sure **Docker Desktop is open and running**.
2. Optional: `copy .env.example .env` (Mac/Linux: `cp`). You only need this if you want to add a CoinGecko key (see below).
3. Start everything. The first build takes a few minutes because it downloads Python and installs packages:

```
docker compose up -d --build
```

4. Check it. Run `docker compose ps -a`. You should see:

Service	Expected state	What it is
db	Up (healthy)	PostgreSQL database
redis	Up (healthy)	Redis message board
migrate	**Exited (0)**	A one-off job that creates the tables, then stops. Exiting is correct.
api	Up (healthy)	The web server (port 8000)

Service	Expected state	What it is
scheduler	Up	Fetches prices every 30 seconds
worker	Up	Builds bars and detects anomalies every 60 seconds

5. Open these in your browser:

URL	What you see
http://127.0.0.1:8000/dashboard/	The live dashboard (prices, charts, anomaly log, latency)
http://127.0.0.1:8000/docs	Interactive API documentation: click any endpoint, then Try it out
http://127.0.0.1:8000/health	Returns {"status":"ok"} when the server is alive

> **What to expect, and what is NOT a bug**
> Prices appear within about 30 seconds. The z-score shows a dash for the first few minutes (it needs at least 2 earlier bars) and stays a rough guess until about 20 minutes of history exist. The forecast says "collecting data" for the first ~10 minutes. Anomalies are **rare** (that is the point), so an empty anomaly list is normal. The browser also logs harmless 404 errors for favicon.ico and .map files; ignore them.

If the scheduler logs "403 Forbidden" from CoinGecko: their free tier sometimes blocks heavy shared IPs. Get a free Demo API key at coingecko.com (developer dashboard), put `COINGECKO_API_KEY=your_key` in the `.env` file in the repo folder, then run `docker compose up -d` again.

6. Stop when done: `docker compose down` (add nothing else; `-v` deletes your data).

1.7 Option B: run the Python code directly (for developers)

Use this when you want to edit code and see changes instantly. Docker still runs the database and Redis, but the three Python programs run on your PC. **Do not run Option A and B at the same time**: they would fight over port 8000 and write duplicate prices.

```
python -m venv venv
venv\Scripts\activate          (Mac/Linux: source venv/bin/activate)
pip install -r requirements-dev.txt
copy .env.example .env      (Mac/Linux: cp)
docker compose up -d db redis
alembic upgrade head
```

A **venv** is a private box of Python packages for this one project, so it does not disturb other projects. **alembic upgrade head** creates the database tables. Now open **three terminals**; in each one, go to the repo folder and activate the venv, then run one command per terminal:

```
python -m app.ingestion.scheduler          # terminal 1: fetch prices
python -m app.analytics.worker             # terminal 2: bars + anomalies
uvicorn app.api.main:app --reload          # terminal 3: web server (--reload restarts on edits)
```

1.8 Run the tests

```
pytest
```

You should see **27 passed**. The tests use a throwaway in-memory database, so they need no Docker, no Redis and no internet. If someone breaks something, a red test tells them immediately.

1.9 Your everyday git routine

```
git checkout main
git pull                                  # get the newest code
git checkout -b yourname/short-description   # your own branch
# ... edit files ...
git status                                # what did I change?
git add .
git commit -m "Add alert when z-score is above 4"
git push -u origin yourname/short-description
```

Then open the link git prints, create the **pull request**, wait for the three green CI checks, get a teammate's review, and click Merge.

- **Never work directly on main.** Always make a branch.
- **Never commit passwords or the .env file.** The .gitignore file already blocks .env; do not fight it.
- **Small commits, clear messages.** "Fix typo in README" is better than "changes".
- **Pull before you start**, so you build on the newest code.

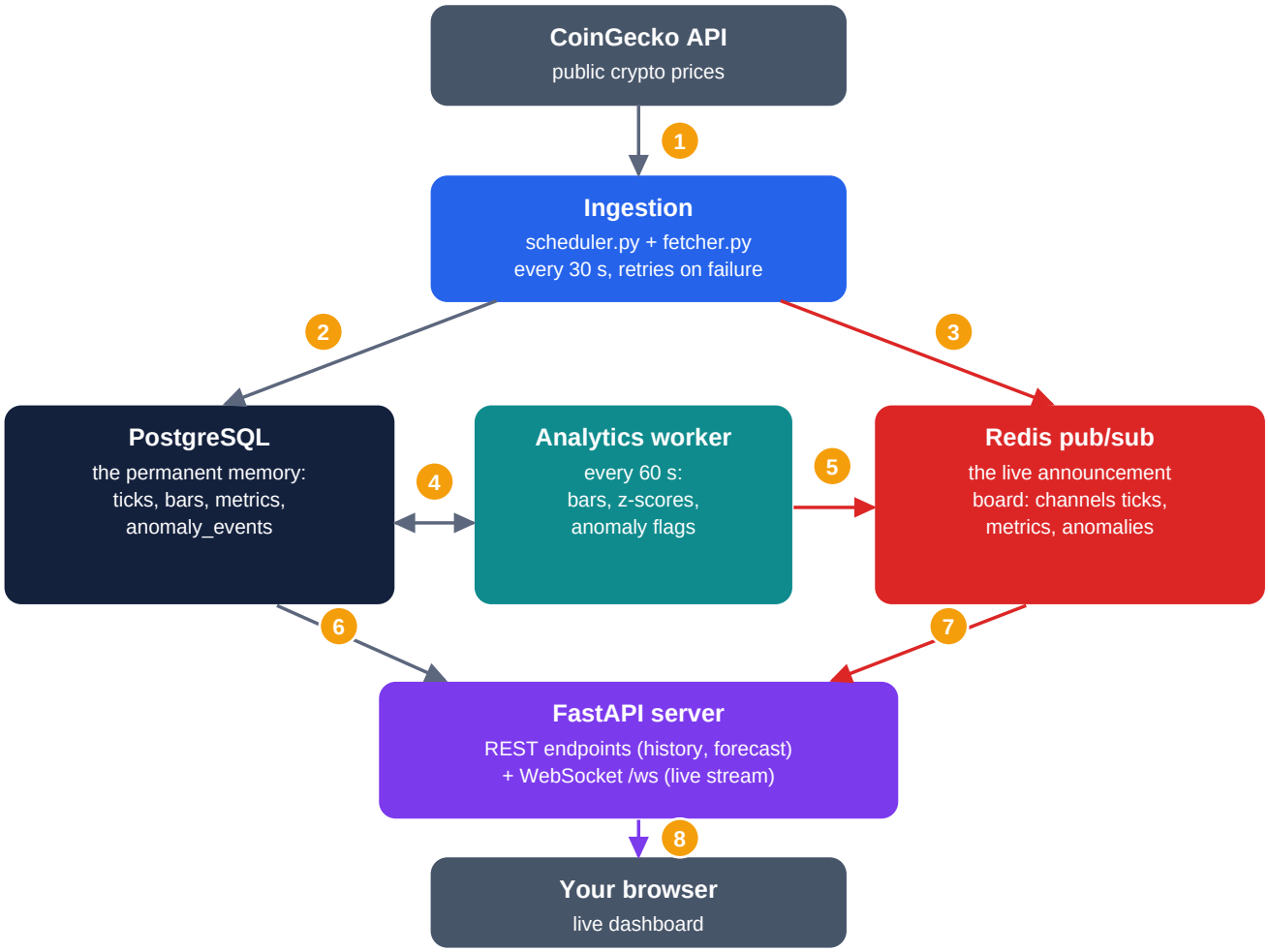
1.10 Troubleshooting: the errors we actually hit

What you see	Why it happens	Fix
open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file	Docker Desktop is not running	Open Docker Desktop, wait for "engine running", retry
Connection refused on port 5432 or 6379	Database or Redis container is not up	`docker compose ps -a`, then `docker compose up -d`
port is already allocated / address in use (8000)	Option A and B running together, or an old process	Stop the other one (Ctrl+C or `docker compose down`)
403 Forbidden from CoinGecko	Free API throttles/blocks some IPs	Add `COINGECKO_API_KEY` to .env (see 1.6)
git: **Password authentication is not supported**	You typed your GitHub password	Use a personal access token instead (1.3)
git: **403 Permission denied**	Invite not accepted, or token has no "repo" box	Accept the invite; create a new token with **repo** ticked
ModuleNotFoundError: No module named 'app...'	Not in the repo root, or a file is missing/was skipped when copying	Run commands from the repo root; check the file exists
Dashboard shows nothing / **Chart is not defined**	app/dashboard/vendor/chart.umd.min.js is missing	It must be committed; re-pull the repo
API says **Not enough data yet**	Fewer than 10 one-minute bars exist	Wait ~10 to 20 minutes with scheduler + worker running
pip fails building **psycopg2**	Old pinned version lacked a Python 3.13 build	Use the repo's pinned requirements (2.9.10 or newer)

2. The flow of the project

2.1 The big picture

Read the diagram top to bottom and follow the numbered circles. Each box is a separate program or service; the arrows are the messages and data passed between them.



#	What happens
1	CoinGecko answers a web request with the latest prices for the five coins.
2	The scheduler saves every price into the **ticks** table in PostgreSQL (the permanent copy).
3	It also **publishes** the price on Redis (an instant announcement, for anyone listening).
4	Once per minute the analytics worker reads recent ticks and writes **bars**, **metrics** and **anomaly events** back to PostgreSQL.
5	The worker publishes each new metric, and any anomaly alert, on Redis.
6	When a browser or tool asks for history or a forecast, the API reads it from PostgreSQL (REST).
7	The API has **one** listener on Redis that hears every announcement.
8	The API forwards every announcement to your browser over the open WebSocket, so charts move live.

2.2 The life of one price

To really understand the system, follow a single price from the internet to your screen:

Moment	What happens	Where in the code
0 s	The scheduler wakes up (it wakes every 30 seconds, forever).	`scheduler.py` run_forever()
0.1 to 1 s	One web request asks CoinGecko for all five coins at once. If it fails, wait 2 s and retry, then 4 s, then give up for this round (logged, not crashed).	`fetcher.py`, `scheduler.py` fetch_with_retry()
next	Each price is saved as a row in the ticks table.	`repository.py` save_tick()
next	The same price is published on Redis channel "ticks".	`publisher.py` publish_tick()
instantly	The API's listener receives it and sends it to every open browser tab.	`broadcaster.py`
instantly	The dashboard updates the price, adds a point to the chart, and measures latency (browser time minus the moment the fetch finished).	`index.html` handleTick()
every 60 s	The worker takes the last *completed* minute of ticks and summarises it as one bar (open, high, low, close).	`aggregator.py`
same cycle	It compares that bar to the previous 20 bars (z-score), saves the metric, and logs an anomaly event if it is extreme.	`stats.py`
same cycle	It announces the metric (and any anomaly) on Redis; the dashboard shows the z badge and the anomaly list.	`worker.py` run_forever()
on request	Someone opens /forecast/bitcoin: the API loads recent bars and projects the next 5 minutes.	`routers/forecast.py`, `forecast.py`

2.3 The database: four tables

A database table is like a spreadsheet: columns describe what is stored and every row is one record. We have four:

Table	One row means...	Written by	Read by
ticks	One price reading of one coin at one moment (asset, price, fetched_at)	scheduler	worker, /prices
bars	One coin's summary for one minute: open, high, low, close	worker (aggregator)	worker, /bars, /forecast
metrics	The statistics for one bar: rolling mean, rolling std, z-score, is_anomaly	worker (stats)	/metrics
anomaly_events	A logged anomaly: when, how extreme (z), at what price	worker (stats)	/anomalies

OHLC means **O**pen (first price in the minute), **H**igh (highest), **L**ow (lowest) and **C**lose (last). Traders have summarised time periods this way for over a century. We use the close for statistics.

> **Raw versus derived data**
> **ticks** are the original evidence and are never edited. **bars**, **metrics** and **anomaly_events** are *derived*: they can always be rebuilt from ticks. That means if we improve the anomaly maths tomorrow, we can recompute everything. If we had only stored the summaries, the original data would be gone forever.

2.4 Three programs running at the same time

Program	Rhythm	Job	If it crashes...
scheduler	every 30 s	Fetch prices, save, announce	No new prices. History, API and dashboard still work.
worker	every 60 s	Bars, z-scores, anomaly flags	Prices keep being collected; analysis catches up on restart.
api (uvicorn)	always on	Answer requests, stream live data	Collection and analysis continue; dashboard goes quiet until restart.

They are separate on purpose: a bug in the maths should never stop price collection. Docker's `restart: unless-stopped` also restarts any of them automatically if it crashes.

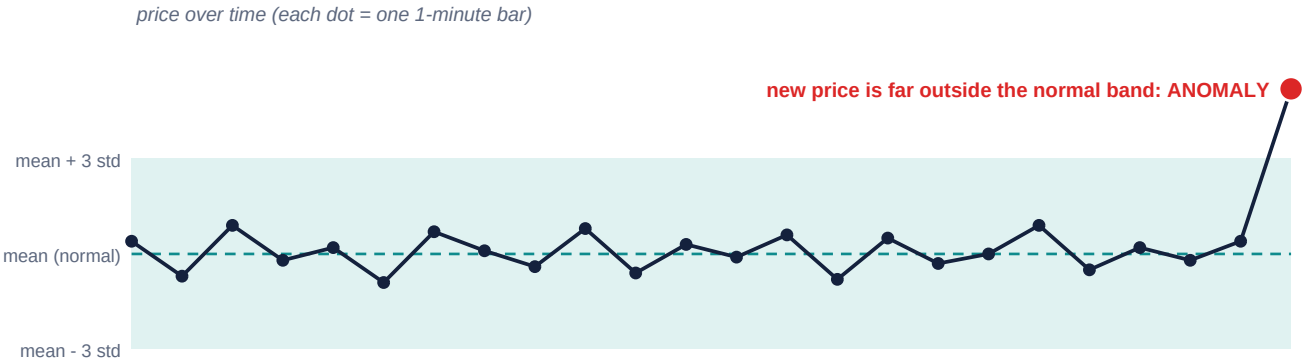
2.5 Redis pub/sub: the radio station

Pub/sub means publish and subscribe. Picture a radio station: the scheduler is a broadcaster that announces "Bitcoin is $82,765" on a channel. It does not know or care who is listening. The API is a listener tuned to the channels. Our three channels are **ticks**, **metrics** and **anomalies**.

Why not let the scheduler talk straight to the browser? Because then the scheduler would need to track every open tab, and every future feature (alerts, a mobile app) would have to be wired into it. With pub/sub, new listeners simply tune in. The trade-off: pub/sub is **fire and forget**. If nobody is listening at that instant, the message is gone. That is fine here because the database holds the permanent history; the dashboard only needs the live stream.

2.6 Anomaly detection with a z-score

Imagine a coin whose price normally wobbles between 99 and 101. A move to 150 is shocking. A move to 101.2 is not. A **z-score** turns that feeling into a number: *how many "typical wobbles" away is the new price?*



The recipe (stats.py): take the closing prices of the **previous 20 bars**. Work out their average (the *mean*) and their typical wobble (the *standard deviation*). Then:

```
z = (newest close - mean of previous 20) / standard deviation of previous 20
```

If the absolute value of z is 3 or more, the bar is flagged as an anomaly. A worked example straight from our tests: the previous 20 closes alternate 100 and 101 (mean 100.5, standard deviation 0.5). A new close of 150 gives z = (150 - 100.5) / 0.5 = **99**. Flagged.

- **The new bar is excluded from its own baseline.** Otherwise a huge spike would stretch the baseline and make itself look less extreme.
- **Only complete minutes are used.** A minute still in progress has no final close yet.
- **Early on, the baseline is thin.** With fewer than 2 earlier bars there is no z-score at all (shown as a dash); from there it is only a rough guess until the full 20-bar window (about 20 minutes) has built up.
- **Flat history means z = 0**, not a crash: dividing by a standard deviation of zero is guarded.
- **Each anomaly is logged once.** Re-processing the same minute never creates duplicates.

> **Honest caveat**
> A threshold of 3 comes from bell-curve statistics, where only about 0.3 percent of readings fall beyond it. Real prices have fatter tails (wild moves happen more often than a bell curve predicts), so treat the flag as "worth a look", not "proof something is wrong".

2.7 The forecast (and why it is honest)

Holt's method keeps two running numbers: a smoothed *level* (where the price is) and a smoothed *trend* (which way and how fast it is moving), then extends that trend forward. It is written by hand in **forecast.py** with no heavy libraries.

But a forecast is only meaningful if it beats a lazy guess. Our baseline is the **naive forecast**: "the price in 5 minutes equals the price now." To compare fairly we run a **walk-forward backtest**: at every minute in history, pretend you only know the past, forecast ahead, and check what actually happened. Accuracy is measured by

MAPE (mean absolute percentage error: on average, how many percent off were we?).

Test data	Holt error	Naive error	Winner
Perfect upward trend (synthetic)	0.0%	4.3%	Holt
Random walk (synthetic)	1.69%	1.33%	Naive
Real Bitcoin, 1-minute horizon	**0.088%**	**0.045%**	**Naive (about 2x better)**

The dashboard therefore labels each forecast **"does not beat naive baseline"** when that is true. Real prices behave close to a random walk, so a trend-chasing model chases noise. Reporting that honestly is the strongest part of this section for an interview.

2.8 How Docker Compose wires it together

The file `docker-compose.yml` describes six services. One shared image (built from the `Dockerfile`) is reused by four of them with a different command each.

Service	Image / command	Port	Notes
db	postgres:16	5432	Data lives in the **pgdata** volume; has a health check
redis	redis:7-alpine	6379	Health check uses redis-cli ping
migrate	our image: alembic upgrade head	-	One-shot job: creates tables, exits. Others wait for it
api	our image: uvicorn app.api.main:app	8000	Serves REST, /ws, and the dashboard files
scheduler	our image: python -m app.ingestion.scheduler	-	Restarts automatically if it crashes
worker	our image: python -m app.analytics.worker	-	Restarts automatically if it crashes

- **Hostnames:** inside Docker, services reach each other by name (`db`, `redis`). On your own PC (Option B) the same things are at `localhost`. The compose file sets the right addresses through environment variables, so the code itself never changes.
- **Start order:** `depends_on` with health checks makes sure the database is truly ready before the migration runs, and the migration finishes before the app starts.
- **One image, many jobs:** building once and reusing it keeps things fast and guarantees every part runs identical code.

3. A tour of every file

Here is the project's folder layout. Folders are shown in bold; every file is explained in the tables that follow.

```
crypto-anomaly-radar/
  Dockerfile  docker-compose.yml  .dockerignore  .gitignore  .env.example
  requirements.txt  requirements-dev.txt  pytest.ini  alembic.ini  README.md
  .github/workflows/ci.yml
  docs/GIT_GUIDE.md
  migrations/        env.py  script.py.mako  versions/0001_create_initial_tables.py
  app/
    config.py
    ingestion/       fetcher.py  scheduler.py
    storage/         database.py  models.py  repository.py
    analytics/       aggregator.py  stats.py  forecast.py  worker.py
    realtime/        publisher.py  broadcaster.py
    api/             main.py  dependencies.py  schemas.py  routers/*.py
    dashboard/       index.html  vendor/chart.umd.min.js
  tests/             conftest.py  test_forecast.py  test_stats.py  test_api.py
```

Suggested reading order for a beginner

config.py, then fetcher.py, scheduler.py, models.py, repository.py, aggregator.py, stats.py, worker.py, forecast.py, publisher.py, broadcaster.py, api/main.py and one router, and finally a test file. That is the same order the data travels. Every folder also contains a file named `__init__.py`: usually empty, it simply tells Python "this folder is a package you can import from".

3.1 Files in the project root

File	What it does	Key things inside
Dockerfile	Recipe for building the app's container image	Python 3.12 slim base, installs requirements first (so Docker can cache that step), copies the code, runs as a normal user, not root
docker-compose.yml	Starts all six services together	db, redis, migrate (one-shot), api, scheduler, worker; health checks; shared settings block named x-app
requirements.txt	Packages the app needs, with exact versions	httpx, sqlalchemy, psycopg2-binary, alembic, python-dotenv, fastapi, uvicorn, redis. Pinned so everyone gets identical versions
requirements-dev.txt	Everything above plus testing tools	Adds pytest; used by developers and CI
pytest.ini	Tells pytest where tests live	testpaths = tests; pythonpath = . so imports like app.x work
alembic.ini	Settings for database migrations	Points to the migrations folder; the real DB address comes from .env
.env.example / .env	Configuration that differs per computer	DATABASE_URL, REDIS_URL, optional COINGECKO_API_KEY. **.env is secret and never committed**; .env.example is the safe template
.gitignore / .dockerignore	Lists files git / Docker must skip	.env, venv, __pycache__, data files
.github/workflows/ci.yml	The CI robot's instructions	Job 1: run pytest on Python 3.12 and 3.13. Job 2: build the Docker stack, check /health and that the dashboard files are served
README.md, docs/GIT_GUIDE.md, CONTRIBUTORS.md	Human-readable docs	Overview, roadmap, quickstart, git cheat sheet, contributor list

3.2 app/ : the heart of the project

config.py and ingestion/ (getting the prices)

File	What it does	Key things inside
config.py	One shared list of coins to track	ASSETS = bitcoin, ethereum, solana, cardano, dogecoin. Change it here and everything (API, dashboard) follows
ingestion/fetcher.py	Asks CoinGecko for prices, **once**, and returns clean data	fetch_prices(assets) returns {coin: {price, fetched_at}}; raises FetchError on any problem; sends browser-like headers (an earlier 403 fix)
ingestion/scheduler.py	Calls the fetcher every 30 s, forever	fetch_with_retry (waits 2 s, 4 s, then gives up politely), run_forever (save then announce each price), asyncio.to_thread for DB writes

storage/ (remembering the prices)

File	What it does	Key things inside
storage/database.py	The single place the database connection is configured	engine, SessionLocal (a factory for short conversations with the DB), Base (parent of all table classes), expire_on_commit=False
storage/models.py	Describes the four tables as Python classes	Tick, Bar, Metric, AnomalyEvent with their columns and indexes (indexes make searches fast)
storage/repository.py	A small helper so other code never touches database sessions	save_tick(asset, price, fetched_at)

analytics/ (understanding the prices)

File	What it does	Key things inside
analytics/aggregator.py	Turns raw ticks into one-minute bars	floor_to_bucket (rounds a time down to its minute), build_bar_for_asset (open/high/low/close), upsert_bar (insert or update, so re-runs are safe)
analytics/stats.py	Rolling statistics and anomaly detection	ROLLING_WINDOW = 20, Z_THRESHOLD = 3.0, compute_metric_for_bar, save_metric (also logs the anomaly once)
analytics/forecast.py	Forecast maths: pure functions, no database	holt_forecast, naive_forecast, mape, backtest (walk-forward comparison)
analytics/worker.py	The loop that ties the analytics together	latest_completed_bucket, run_cycle (one pass over all coins; returns plain dicts), run_forever (runs it each minute, then publishes results)

realtime/ (telling the browser immediately)

File	What it does	Key things inside
realtime/publisher.py	Announces events on Redis	publish_tick, publish_metric, publish_anomaly; channel names ticks / metrics / anomalies
realtime/broadcaster.py	Receives announcements and forwards them to browsers	ConnectionManager (the set of open WebSockets), redis_listener (one listener per server, started at boot)

api/ (answering questions)

File	What it does	Key things inside
api/main.py	Creates the web application and plugs in the pieces	FastAPI app, lifespan (starts the Redis listener at boot), CORS, all routers, serves the dashboard at /dashboard
api/dependencies.py	Gives each request its own database session	get_db(): opens a session, hands it over, always closes it afterwards
api/schemas.py	Defines exactly what API responses look like	Pydantic models: TickOut, BarOut, MetricOut, AnomalyEventOut. Separate from the DB classes on purpose
api/routers/assets.py	GET /assets	The list of tracked coins
api/routers/prices.py	GET /prices/{coin}	Recent raw ticks, newest first (limit 1 to 1000)
api/routers/bars.py	GET /bars/{coin}	Recent one-minute bars
api/routers/metrics.py	GET /metrics/{coin}	Recent z-scores and flags
api/routers/anomalies.py	GET /anomalies	The anomaly log, optionally filtered by coin
api/routers/forecast.py	GET /forecast/{coin} and /forecast/{coin}/accuracy	The 5-bar forecast, and the Holt-versus-naive backtest. Needs at least 10 bars
api/routers/ws.py	WebSocket /ws	Registers the browser with the ConnectionManager and waits until it disconnects

Unknown coins return a **404** error and out-of-range values (like limit=0) return **422**, so clients get clear messages instead of crashes.

dashboard/ (what you see)

File	What it does	Key things inside
dashboard/index.html	The entire live dashboard in one file (HTML + CSS + JavaScript)	buildCards, handleTick / handleMetric / handleAnomaly, trackLatency (the 71 ms number), automatic WebSocket reconnect, refreshForecast every 60 s
dashboard/vendor/chart.umd.min.js	The Chart.js library, saved locally	Kept in the repo on purpose: the page works even if a CDN website is blocked or down (this once broke the dashboard)

3.3 migrations/ : version control for the database

Code changes are tracked by git. **Alembic** tracks changes to the database's *structure* the same way.

File	What it does
migrations/env.py	Alembic's startup script: loads our models and reads DATABASE_URL from the environment
migrations/script.py.mako	A template Alembic uses when generating future migration files
migrations/versions/0001_create_initial_tables.py	Migration #1: creates the four tables and indexes (upgrade), or removes them (downgrade)

3.4 tests/ : the safety net

File	Tests	What it checks
tests/conftest.py	(setup)	Builds a throwaway in-memory SQLite database for every test, so tests never touch your real data
tests/test_forecast.py	10	Forecast maths: trend extension, naive baseline, MAPE, backtest counts, and that the random-walk case does not beat naive
tests/test_stats.py	7	Spikes are flagged; the baseline excludes the current bar (z = 99 exactly); flat history does not crash; anomalies logged once
tests/test_api.py	10	Endpoints: 404 for unknown coins, validation, newest-first ordering, filters, "not enough data" refusal, forecast shape and accuracy

Total: **27 tests**. CI runs them on every pull request on two Python versions.

4. Design and implementation decisions

Every choice below had alternatives. Knowing *why* you chose something, and what it costs, is what separates "I followed a tutorial" from "I built this". Each entry gives the decision, the reason, and the trade-off.

4.1 Structure and reliability

1. Split every job into its own small layer

What we did: The fetcher only fetches; the scheduler only decides when and retries. The aggregator only builds bars; stats only does maths; the worker only orchestrates.

Why: Each piece is easy to understand, test and replace. It also let two people build in parallel on different files with zero merge conflicts.

Trade-off: More files and some jumping around while reading.

2. Async code for ingestion

What we did: The scheduler uses asyncio and httpx instead of simple blocking code.

Why: Most of the time is spent waiting for the network. Async lets one program wait without freezing, and it is the natural fit for the live-streaming parts too.

Trade-off: Async is harder to reason about, and it does not mix freely with normal (blocking) libraries; see the next decision.

3. Database writes are pushed to a worker thread

What we did: asyncio.to_thread(save_tick, ...) runs each blocking database call outside the main async loop.

Why: A blocking call inside an async loop freezes everything else until it returns. The thread keeps the loop responsive.

Trade-off: Fully async database drivers exist, but they add complexity we do not need at five writes every 30 seconds.

4. Fail loudly, retry politely, never crash the loop

What we did: A bad fetch raises **FetchError**. The scheduler retries after 2 s then 4 s (**exponential backoff**), then logs and skips that round.

Why: Silently returning empty data would quietly corrupt the dataset. Waiting longer between retries gives a struggling server room to recover. And one bad response must not kill days of collection.

Trade-off: A short gap in data during an outage. Gaps are visible and honest, which is better than fake values.

5. Three separate programs (scheduler, worker, API)

What we did: Each runs as its own process and its own container.

Why: A bug in the analytics must not stop price collection. Each can be restarted, scaled or replaced alone, and Docker restarts crashed ones automatically.

Trade-off: More moving parts to start, which is why Docker Compose exists.

4.2 Data and storage

6. PostgreSQL, with raw data kept separate from derived data

What we did: ticks are stored untouched; bars, metrics and anomaly_events are computed from them.

Why: Derived tables can be rebuilt if the maths improves. Raw data lost is lost forever. Postgres is also the most requested database in job listings.

Trade-off: Some duplication of information, and slightly more storage.

7. SQLAlchemy models and Alembic migrations (not create_all)

What we did: Tables are Python classes, and structure changes are versioned files in migrations/.

Why: On a database that already holds data, you cannot just "recreate everything". Migrations are reviewable in a pull request, repeatable on any machine, and reversible. This is how real teams change schemas.

Trade-off: More setup than a one-line create_all.

8. A thin repository layer and separate API schemas

What we did: The scheduler calls **save_tick()** and never sees a database session. API responses use Pydantic schemas, not the DB classes.

Why: Storage details can change without touching ingestion code, and internal column changes do not accidentally become breaking API changes.

Trade-off: A little extra code per feature.

9. Sessions that survive commit, and plain values across boundaries

What we did: SessionLocal uses **expire_on_commit=False**, and the worker's run_cycle returns plain dictionaries, not ORM objects.

Why: By default SQLAlchemy invalidates objects after commit, and reading one after its session closed raised **DetachedInstanceError** (a real bug we hit). Short-lived sessions plus plain values avoid the whole class of problem.

Trade-off: Objects are not auto-refreshed, which is fine because we never reuse them across sessions.

4.3 Analytics and real-time

10. One-minute bars, and only *completed* minutes

What we did: Ticks are rolled into OHLC bars. latest_completed_bucket() skips the minute still in progress.

Why: A minute that has not finished has no true close, so statistics on it would be wrong and would change on every refresh.

11. A plain z-score instead of machine learning

What we did: 20-bar rolling window, threshold 3, baseline excludes the current bar, zero-variance guarded, each anomaly logged once.

Why: You can explain exactly why any flag fired ("3.4 standard deviations from the last 20 minutes"). A black-box model would be harder to trust and to defend.

Trade-off: A simple method misses seasonal or multi-coin patterns. Section 6 lists better options.

12. Redis pub/sub with one listener per server process

What we did: Producers publish; a single background task in the API subscribes once and fans messages out to all connected browsers.

Why: Producers never track clients. Ten browser tabs cost one Redis subscription, not ten. It also keeps working correctly if you later run several API servers, because each one listens to Redis on its own.

Trade-off: Pub/sub is fire-and-forget: no replay for someone who connects late. Acceptable here because history lives in Postgres.

13. WebSocket instead of repeatedly asking the server

What we did: The browser opens one WebSocket and the server pushes updates; the page reconnects automatically if the link drops.

Why: Polling wastes requests and adds delay. Push is simpler to reason about for live data and gives lower latency.

14. A hand-written forecast, judged against a naive baseline

What we did: Holt's smoothing in about 20 lines of Python, scored by a walk-forward backtest against "price stays the same".

Why: Heavy forecasting libraries pulled in compiled packages that had caused install failures on new Python versions, and a hand-written version can be explained line by line. The baseline comparison is what makes the accuracy claim honest.

Trade-off: It is a simple model. The honest finding: it does not beat the naive guess at a 1-minute horizon.

4.4 Packaging, testing and delivery

15. Docker Compose with one image and many commands

What we did: The Dockerfile builds one image; the api, scheduler, worker and migrate job each run a different command. Health checks and depends_on control start order; a named volume keeps the data; the app runs as a non-root user.

Why: One command starts everything, identically on any PC, and the same image is used everywhere so there is no "works on my machine".

16. Tests that need nothing installed

What we did: Tests use an in-memory SQLite database built from the same models, and override the API's database dependency. The test client skips the app's startup, so no Redis is required.

Why: Fast (about 2 seconds), runnable anywhere including CI, and they cannot damage real data.

Trade-off: SQLite is not identical to Postgres, so a Postgres-specific bug could slip through; the Docker smoke test in CI covers the real stack.

17. CI on two Python versions plus a Docker smoke test

What we did: Every pull request runs the tests on Python 3.12 and 3.13, builds the real stack, calls /health, and fetches the dashboard and its Chart.js file.

Why: A red X on a pull request catches problems before they reach main. The last check exists because a missing file once broke the dashboard.

18. Chart.js saved inside the repo (vendored), versions pinned

What we did: We keep chart.umd.min.js locally and pin every dependency to an exact version.

Why: A third-party website being blocked or slow should not break the demo, and exact versions mean everyone installs the same thing.

Trade-off: The repo is slightly larger; upgrades are manual.

19. Honest scope: no public live deployment

What we did: We chose reproducibility (one command) over a hosted demo.

Why: Free hosting tiers put services to sleep after inactivity (a sleeping service cannot collect prices) and delete free databases after about a month. A dead demo link looks worse than none.

20. Honest numbers

What we did: The 71 ms latency is labelled "measured locally": the dashboard subtracts the fetch timestamp from the browser's clock, which is accurate only because both are on the same computer.

Why: Every figure on the resume can be reproduced and defended. If a number cannot be explained, it should not be quoted.

4.5 Bugs we hit, and what each one taught

Bug or problem	Root cause	Lesson
DetachedInstanceError in the worker	Reading an ORM object after its session closed	Return plain values; expire_on_commit=False
403 Forbidden from CoinGecko	Free API blocks traffic that looks automated	External services can refuse you: send proper headers, support an API key, retry
"Chart is not defined"	The CDN script did not load	Do not depend on third-party hosts for a demo; vendor the file; add a CI check
Chart crashed on pointRadius	Chart.js evaluates options at creation, before our state existed	Initialise data before constructing the thing that reads it
pip could not build psycopg2	No prebuilt package for Python 3.13 in the old pinned version	Check compatibility when pinning; prefer prebuilt wheels
ModuleNotFoundError after copying files	Files were skipped or deleted while unzipping	A clean-clone build (Docker/CI) proves nothing is missing
Duplicate "redis" key in docker-compose.yml	Two edits both added the service	Validate config files; run docker compose config
git 403 despite being invited	Token created with no scopes ticked	Read permission errors literally: authentication worked, authorisation did not

5. Interview questions and model answers

These are the questions a recruiter or interviewer could realistically ask about this project. The answers are written in plain language: **do not memorise them word for word**. Understand the idea, then say it in your own voice. A good pattern is: give the simple answer in one sentence, then add one detail, then mention a trade-off.

Numbers you can quote (all true and reproducible)

Fact	Value
Coins tracked / polling interval	5 coins, every 30 seconds
Bar size / anomaly baseline	1-minute bars, previous 20 bars, flag at |z| of 3 or more
Latency (fetch to browser)	About 71 ms average, measured locally on one machine
Tests / CI	27 automated tests; CI on Python 3.12 and 3.13 plus a Docker smoke test
Forecast result (Bitcoin, 1 minute ahead)	Holt error 0.088% versus naive 0.045%: the naive guess won
Services in Docker Compose	6 (db, redis, migrate, api, scheduler, worker)

If you used AI tools to help build this

Say so calmly. Interviewers use Copilot and similar tools too. What matters is that **you can explain every file, change it, and debug it**. Use this handbook until that is true, and practise explaining the bugs in section 4.5 out loud. Never claim work you cannot explain.

A. The project overall

1. Walk me through the project in about a minute.

"It is a real-time monitoring pipeline for crypto prices. A Python program polls a public API every 30 seconds and stores the prices in PostgreSQL. A second program turns them into one-minute bars and compares each new bar to the previous 20 using a z-score, flagging statistical outliers as anomalies. A FastAPI server exposes the history over REST and streams live updates to a browser dashboard over a WebSocket, using Redis pub/sub in between. It is containerised with Docker Compose and covered by 27 tests and CI."

2. What problem does it solve, and who would use it?

It is a learning and portfolio project, but it mirrors real monitoring systems: fraud detection, server monitoring and trading surveillance all do "learn what normal looks like, alert when something is not." A real user could be someone watching a basket of coins who wants to know the moment one behaves unusually.

3. What was the hardest part?

Pick a real one. A good choice is the **DetachedInstanceError**: the worker crashed when reading a database object after its session had closed. I learned how SQLAlchemy sessions expire objects on commit, fixed it by returning plain values and disabling expiry, and avoided the same bug in the API layer. (Details in 4.5.)

B. Architecture

4. Why three separate programs instead of one script?

Failure isolation and independence. If the analytics code has a bug, price collection keeps running. Each part can be restarted or scaled on its own, and Docker restarts crashed ones automatically. The cost is more moving parts, which Docker Compose manages with one command.

5. What happens if CoinGecko is down or rate-limits you?

The fetcher raises a clear error, and the scheduler retries after 2 seconds and then 4 seconds. If it still fails it logs the problem and tries again at the next cycle instead of crashing. There is a visible gap in the data rather than fake values. We actually hit a 403 block once, fixed with proper headers and an optional API key.

6. Why PostgreSQL? Why not a file, SQLite or MongoDB?

The data is structured and time-based, and I want reliable queries, indexes, and safe concurrent writes from several programs at once. SQLite does not handle multiple writer processes well, and a document database adds no value for rows with fixed columns. Postgres is also the industry default. (I do use SQLite, but only in tests.)

7. Why store raw ticks AND bars and metrics?

Raw ticks are the evidence and never change. Bars, metrics and anomalies are derived and can be recomputed if my maths improves. If I had stored only summaries, I could never rebuild them. This raw-versus-derived layering is a common data-engineering pattern.

C. Python and async

8. What is async, and why use it here?

Normally a program waits idle while a slow thing (a network call) finishes. Async lets one program switch to other work while it waits, using **await** to mark waiting points. Fetching prices and talking to Redis are mostly waiting, so async fits. It is not parallel computation; it is efficient waiting.

9. What is asyncio.to_thread and why did you use it?

My database library is blocking: while it runs, nothing else in the async loop can run. **to_thread** runs that call in a separate worker thread and lets the loop keep going. It is the simple bridge between blocking and async code.

10. Process vs thread vs coroutine, in simple terms?

A **process** is a separate running program with its own memory (our scheduler, worker and API are three processes). A **thread** is a worker inside one program. A **coroutine** (async function) is a task that can pause and resume, all within one thread.

11. What is dependency injection in FastAPI? (Depends(get_db))

Instead of every endpoint creating its own database session, it declares what it needs and FastAPI supplies it, and cleans up afterwards. It keeps endpoints short, guarantees the session is closed, and lets tests swap in a different database, which is exactly what our API tests do.

D. Database

12. What is an index, and where did you use one?

An index is like a book's index: it lets the database find rows without scanning the whole table. We index ticks by (asset, time) because every query asks "recent rows for one coin". On bars we add a **unique** index on (asset, bucket_start), so the same coin and minute can never be stored twice.

13. What is a migration, and why Alembic?

A migration is a versioned script that changes the database structure, like git for the schema. It is repeatable on any machine, reviewable in a pull request, and reversible. Calling create_all is fine once, but cannot safely evolve a database that already contains data.

14. What does "idempotent" mean, and do you have an example?

Doing something twice has the same effect as doing it once. Our **upsert_bar** updates an existing bar instead of inserting a duplicate, and save_metric logs an anomaly only once, so re-processing a minute is harmless.

E. Real-time

15. WebSocket vs polling vs Server-Sent Events?

Polling asks repeatedly (wasteful and delayed). A WebSocket is one long-lived two-way connection, so the server pushes the moment data exists. Server-Sent Events are one-way push over normal HTTP, simpler but only server to client. We chose WebSocket; since we only send data one way, SSE would also have worked.

16. What does Redis do here, and what are the limits of pub/sub?

Redis is a fast message board. Producers publish; the API listens once and forwards to browsers. The limit: pub/sub has **no memory**. A listener who is offline misses messages. For guaranteed delivery and replay you would use Redis Streams or Kafka. Here that is acceptable because the database holds the history.

17. How did you measure the 71 ms latency, and is it trustworthy?

The dashboard subtracts the fetch timestamp from the browser's current time for every tick and averages it. That is only valid because the scheduler and browser share one computer's clock. Across machines, clock differences would distort it, so I label it "measured locally", and across the internet it would be higher.

18. What if 1,000 people opened the dashboard?

One API process can hold many WebSocket connections, but it sends to each in turn, so at larger scale I would run several API processes behind a load balancer. That already works, because each process subscribes to Redis itself. Beyond that: a dedicated WebSocket gateway and load testing.

F. Statistics and the forecast

19. What is a z-score, and why a threshold of 3?

It is the distance from the average measured in standard deviations (typical wobble sizes). In a bell-curve world only about 0.3% of values are beyond 3, so it signals "unusual". Real prices have fatter tails, so 3 is a practical starting point I would tune using labelled historical events.

20. Why exclude the current bar from its own baseline?

If the spike is included, it inflates the average and the spread, making itself look less extreme. Our test proves it: with 20 calm bars and a jump to 150, the z-score is exactly 99, which is only possible if the baseline is calm.

21. How would you know if your anomaly detector is any good?

Label a set of real events (true spikes and calm periods), then measure **precision** (of the flags I raised, how many were real?) and **recall** (of the real events, how many did I catch?). Then tune the window and threshold. I have not done this formally yet, and I would say so; it is on the roadmap.

22. Is your forecast any good?

"Honestly, no, and I can prove it." Over one minute, Bitcoin's naive baseline (price stays put) had 0.045% average error versus 0.088% for my Holt model. Short-term prices behave like a random walk, so a model that extrapolates a trend just chases noise. The value of the project is the backtesting harness that reveals this, and the dashboard labels the forecast "does not beat naive" instead of hiding it.

23. What is a walk-forward backtest, and why not a random train/test split?

For time series, the future must never leak into the past. Walk-forward trains on data up to time t, forecasts t+1, then moves forward one step and repeats. A random split would let the model "see" later data and look unrealistically good.

24. What is MAPE and what are its weaknesses?

Mean absolute percentage error: on average, how many percent off. It is easy to read, but it breaks when the true value is near zero and treats over- and under-predictions asymmetrically. For prices far from zero it is fine, and I skip any zero values.

G. Testing, Docker and CI

25. How do you test code that needs a database and Redis?

Tests use a throwaway in-memory SQLite database built from the same models, and I override the API's database dependency. The test client does not run the app's startup, so no Redis is needed. Tests take about two seconds. The weakness is SQLite is not Postgres, which is why CI also boots the real Docker stack.

26. What does your CI do?

On every pull request, GitHub Actions runs all 27 tests on Python 3.12 and 3.13, then builds the Docker stack, waits for /health, and checks the dashboard and its JavaScript file are being served. It catches missing dependencies and missing files before anyone merges.

27. Image vs container? What do health checks and depends_on do?

An image is the packaged recipe; a container is a running copy of it. A **health check** is a command Docker runs to see if a service is really ready (not just started). With **depends_on** conditions, the migration waits for a healthy database, and the app waits for the migration to finish.

H. Honesty, limits and growth

28. What are the limitations of the project?

Be direct: it runs on one machine; it polls a free API every 30 seconds (not true tick data); pub/sub has no replay; the anomaly method is simple and not formally evaluated; the forecast does not beat the baseline; there is no authentication, alerting, or monitoring of the pipeline itself. Then add: "Section 6 lists how I would address each."

29. What would you do differently, or next?

First, make price saving idempotent with a unique constraint. Then add alerts (Slack webhook) with a cooldown, Prometheus metrics with a Grafana dashboard, and evaluate the detector with labelled events. Later, use exchange WebSocket feeds and a time-series database.

30. Tell me about working with a teammate (git workflow).

We never worked directly on main. Each person used a branch, opened a pull request, and CI had to pass before merging. We split work by file with an agreed interface up front (for example, what fetch_prices returns) so we never conflicted. Early on we debugged GitHub token permissions and collaborator invites, which taught us to read permission errors literally.

6. Future scope: toward an industry-level system

Today the project is a solid, well-tested **single-machine** system. Real companies run the same idea at much larger scale and with far stricter reliability. This section shows the gap and a realistic order for closing it. None of these are things we have already built, so do not claim them on a resume until they exist.

6.1 Where we are versus industry level

Area	Today	Industry level
Data source	One free REST API polled every 30 s	Exchange WebSocket feeds (true tick data), several sources, cross-checking and fallbacks
Ingestion safety	Retries inside the program	Message queue, idempotent writes (unique keys), dead-letter queue for bad messages
Storage	Plain PostgreSQL tables	TimescaleDB or partitioned tables, retention rules, pre-computed rollups
Streaming	Redis pub/sub (no replay)	Redis Streams or Kafka: durable, replayable, many consumers
Anomaly detection	Rolling z-score, threshold 3	Robust methods (median/MAD, EWMA), seasonality, multi-coin logic, ML models, measured precision and recall
Forecasting	Holt smoothing, 1 model	Damped trend, ARIMA/Prophet/ML, prediction intervals, accuracy tracked per horizon
Alerting	A flag on the dashboard	Slack / email / webhook / pager alerts with cooldowns and de-duplication
API	Open, no limits	Authentication (API keys or JWT), rate limiting, pagination, versioned paths (/v1)
Observability	Plain log lines	Structured logs, Prometheus metrics, Grafana dashboards, tracing, alerts on the pipeline itself
Security	Demo passwords in the compose file	Secrets manager, least-privilege accounts, TLS, dependency vulnerability scanning
Deployment	Docker Compose on a laptop	Managed cloud (ECS or Kubernetes), infrastructure-as-code (Terraform), staging and production, rolling releases
Testing	27 unit and API tests on SQLite	Integration tests on real Postgres and Redis, load tests, contract tests, coverage gates
Frontend	One HTML file	A React/TypeScript app with settings, user accounts and alert-rule editing
Data quality	Implicit	Automatic checks for gaps, stale data and outliers (for example Great Expectations)

6.2 A realistic roadmap

Level 1: small upgrades (a day or a weekend each)

- **Unique constraint on ticks** plus insert-or-ignore, so a retry can never store a price twice.
- **Configurable settings** (threshold, window, interval) through environment variables instead of constants.

- **Slack webhook alert** on a new anomaly, with a cooldown so one move does not cause ten messages.
- **Structured JSON logging** and a **/metrics** endpoint for Prometheus.
- **Code quality in CI:** ruff (lint), black (format), mypy (types), and a test coverage report.
- **README polish:** architecture diagram, screenshots or a demo GIF, and the CI badge.

Level 2: real engineering (a few weeks)

- **Binance or Coinbase WebSocket ingestion** for true tick data, with automatic reconnection.
- **TimescaleDB** (a Postgres extension for time series): hypertables, compression, retention and continuous aggregates replace our hand-built bars.
- **Better anomaly detection** (median and MAD, EWMA) and a **labelled evaluation** reporting precision and recall.
- **Better forecasting:** damped trend, per-horizon scoring, prediction intervals; show forecasts only when they earn it.
- **API keys and rate limiting**, pagination and a /v1 prefix.
- **Integration tests** running against real Postgres and Redis containers in CI, and the image published to a registry.
- **Grafana dashboards** for the pipeline's own health: fetch failures, lag, queue depth.

Level 3: production scale (months, team-sized)

- **Kafka or Redis Streams** between ingestion and analytics for durability and replay.
- **Cloud deployment** with Terraform, a managed database, secrets in a vault, staging and production, rolling releases.
- **Autoscaling** the API and WebSocket layer; load tests proving the numbers.
- **Multi-asset intelligence:** correlation between coins, market-wide regime detection, ML anomaly models tracked with MLflow.
- **User accounts and an alert-rule builder** in a React app.
- **Reliability practice:** service-level objectives, on-call runbooks, incident reviews, cost monitoring.

> **A good next three if you only do three**
> 1) Idempotent ticks with a unique constraint. 2) A Slack alert with cooldown. 3) Prometheus metrics plus a Grafana dashboard. Together they turn a demo into something that *behaves* like a service: safe to retry, tells a human when something happens, and can be monitored.

6.3 Cautions for the real world

- **This is not financial advice.** Never present anomaly flags or forecasts as trading signals.
- **Check data provider terms.** Free APIs limit commercial use and request rates; production systems pay for licensed feeds.
- **Do not overclaim.** "Built a monitoring pipeline" is true. "Built a trading system" is not.
- **Hosting:** free tiers sleep services and expire databases, so a live demo needs a paid always-on plan or your own server.

6.4 A learning path if you are new

Order	Learn	Where it appears in this project
1	Git and GitHub basics	Branches, pull requests, CI results
2	Python fundamentals, functions, modules	Every file under app/
3	SQL basics (SELECT, WHERE, JOIN, INDEX)	Tables, queries, psql commands
4	How the web works: HTTP, JSON, REST	fetcher.py, API routers
5	Docker basics	Dockerfile, docker-compose.yml
6	async/await in Python	scheduler.py, worker.py, broadcaster.py
7	FastAPI and Pydantic	app/api/
8	Testing with pytest	tests/
9	Basic statistics: mean, standard deviation	stats.py, forecast.py

Good free starting points: the official **FastAPI tutorial**, Docker's **Get Started** guide, the **PostgreSQL tutorial**, and the **pytest** documentation.

Appendix A: Glossary

Term	Meaning
API	A set of doors another program can knock on to ask for data or actions.
REST	A common style of API using web addresses (like /bars/bitcoin) and simple verbs such as GET.
Endpoint	One specific address of an API, e.g. /health.
JSON	A text format for data, like {"asset": "bitcoin", "price": 82765}.
WebSocket	A connection that stays open so the server can push data to the browser instantly.
async / await	A way of writing code that can pause while waiting and let other work proceed.
Event loop	The scheduler inside asyncio that decides which paused task runs next.
Thread / Process	A worker inside a program / a separate running program.
Container	A lightweight sealed box running one program with everything it needs (Docker).
Image	The frozen blueprint a container is started from.
Volume	Storage that survives when a container is removed.
Docker Compose	A tool to start several containers together from one file.
Health check	A command Docker runs to confirm a service is actually working.
Migration	A versioned script that changes the database structure.
ORM	A tool (SQLAlchemy) mapping database tables to Python classes.
Session	One short conversation with the database.
Index	A lookup structure that makes searching a table fast.
Idempotent	Safe to run twice: the second run changes nothing.
Pub/sub	Publish and subscribe: senders announce on channels; listeners tune in.
Tick	One price reading at one moment.
Bar / OHLC	A summary of one time period: Open, High, Low, Close.
Anomaly	A value unusually far from what is normal.
Mean	The average.
Standard deviation	How far values typically wander from the average.
Z-score	How many standard deviations a value is from the average.
Rolling window	Always the most recent N items (here, the last 20 bars).
Baseline (naive)	The laziest sensible guess, used as a yardstick: "the price stays the same".
Backtest	Testing a method on past data pretending you did not yet know the future.
MAPE	Mean absolute percentage error: average percent the forecast was off.

Term	Meaning
Latency	Delay between something happening and you seeing it.
CI	Continuous integration: a robot that tests every change automatically.
Pull request	A proposal to merge your branch into main, reviewed by teammates.
Personal access token	A generated password for git/GitHub use.
Environment variable / .env	Configuration kept outside the code; .env holds secrets locally and is never committed.
Pinned version	Requiring an exact package version so everyone gets identical behaviour.
venv	A private folder of Python packages for one project.

Appendix B: Command cheat sheet

Task	Command
Download the project	`git clone https://github.com/jayasinghania/crypto-anomaly-radar.git`
Get newest code	`git checkout main` then `git pull`
Start your own branch	`git checkout -b yourname/what-you-do`
See what changed	`git status`
Save a snapshot	`git add .` then `git commit -m "message"`
Upload your branch	`git push -u origin yourname/what-you-do`
Start everything (Docker)	`docker compose up -d --build`
See service states	`docker compose ps -a`
Watch a service's logs	`docker compose logs -f scheduler`
Stop (keeps data)	`docker compose down`
Stop and WIPE data	`docker compose down -v` **(danger)**
Start only DB + Redis	`docker compose up -d db redis`
Create / update tables	`alembic upgrade head`
Run tests	`pytest` or `pytest -v`
Run scheduler locally	`python -m app.ingestion.scheduler`
Run worker locally	`python -m app.analytics.worker`
Run API locally	`uvicorn app.api.main:app --reload`

Looking inside the database

Open a SQL prompt inside the database container:

```
docker exec -it crypto_pulse_db psql -U crypto_pulse -d crypto_pulse
```

Then try these queries (type `\q` to leave):

```
SELECT COUNT(*) FROM ticks;
SELECT asset, price, fetched_at FROM ticks ORDER BY id DESC LIMIT 5;
SELECT asset, bucket_start, close FROM bars ORDER BY bucket_start DESC LIMIT 5;
SELECT asset, bucket_start, z_score, is_anomaly FROM metrics
  WHERE z_score IS NOT NULL ORDER BY bucket_start DESC LIMIT 5;
SELECT * FROM anomaly_events ORDER BY ts DESC LIMIT 10;
```

Appendix C: Addresses and defaults

Thing	Value	Notes
Dashboard	http://127.0.0.1:8000/dashboard/	Live view
API docs	http://127.0.0.1:8000/docs	Interactive; try endpoints in the browser
Health	http://127.0.0.1:8000/health	Should return status ok
Forecast examples	/forecast/bitcoin and /forecast/bitcoin/accuracy	Need about 10+ bars of data
PostgreSQL	localhost:5432, user / password / database: crypto_pulse	Local demo credentials only; never reuse them anywhere real
Redis	localhost:6379	No password in the demo
Container names	crypto_pulse_db, _redis, _api, _scheduler, _worker, _migrate	Used with docker exec / logs